

RedMoon – Memoria del Proyecto

Ingeniería del Software I – Curso 2025–2026 Entrega Final

Realizado por **Lukas Otero Asumendi**



1. Introducción

El proyecto **RedMoon** es un juego multijugador online basado en roles ocultos (Lobo, Aldeano, Vidente, Bruja, etc.) y comunicación en tiempo real. El objetivo del proyecto ha sido desarrollar una aplicación web accesible desde navegador, con salas privadas, sincronización mediante WebSockets y una interfaz que guía al jugador a través de las fases del juego.

Esta memoria recoge el proceso completo de desarrollo del proyecto, incluyendo la aplicación de la metodología Scrum, la evolución del backlog, las decisiones técnicas tomadas, la gestión del proyecto y el trabajo realizado por cada miembro del equipo. El objetivo no es únicamente mostrar el producto final, sino también el camino que ha llevado a él.

2. Scrum

2.1. Estructura y funcionamiento del equipo Scrum

En este caso el equipo estaba compuesto por mí únicamente, por lo que he sido yo el responsable de todos los roles.

2.2. Sprint Reviews

Sprint 1 — Sprint Review

En este sprint se ha construido toda la base técnica del proyecto. Se implementó la creación de salas privadas con código único y la unión mediante dicho código, garantizando que los jugadores puedan conectarse entre sí de forma estable. La comunicación en tiempo real mediante WebSockets quedó completamente funcional, permitiendo actualizar la lista de jugadores conectados sin recargar la página.

Además, se desarrolló una interfaz mínima que permite probar el flujo básico del juego: introducir nombre, crear sala, unirse y visualizar los jugadores conectados. Este sprint también sirvió para definir la arquitectura del servidor, los modelos iniciales y la estructura de comunicación cliente-servidor que se utilizará en los siguientes sprints.

El resultado es una base sólida sobre la que se puede construir el resto del juego.

Sprint 2 — Sprint Review

En este sprint se implementó el sistema de roles y la preparación de partida. El servidor asigna automáticamente roles como Lobo, Aldeano, Vidente o Bruja, y envía esta información de forma privada a cada jugador mediante canales individuales. La interfaz se actualizó para mostrar cuándo la partida está lista para comenzar, y se añadió el control de host, garantizando que solo el creador de la sala pueda iniciar la partida.

Este sprint permitió establecer el flujo inicial del juego y dejó todo preparado para introducir la lógica de fases (noche/día) en el siguiente sprint.

Sprint 3 — Sprint Review

Este sprint implementa la lógica principal del juego. Durante el turno de noche, los roles especiales realizan sus acciones: los lobos eligen víctima, la vidente inspecciona y la bruja actúa. Durante el día, se muestra la víctima y se inicia una votación pública para linchar a un jugador.

El servidor controla automáticamente el cambio entre fases y los temporizadores, garantizando que la partida avance sin intervención manual. Se realizaron pruebas intensivas para asegurar que la comunicación entre cliente y servidor fuera fluida y que las acciones de los roles se sincronizaran correctamente.

El resultado es un flujo de juego completo, coherente y funcional.

Sprint 4 — Sprint Review

En este sprint se completó toda la interfaz del juego. Cada fase tiene ahora su propia pantalla: lobby, roles, noche, día y resultados. Se añadieron animaciones básicas, indicadores visuales de fase y feedback para acciones como seleccionar víctima, votar o confirmar decisiones.

También se implementó el sistema multilingüe, permitiendo cambiar el idioma del juego y adaptando todos los textos de las pantallas. Además, se añadieron roles especiales adicionales y se ajustaron sus habilidades dentro de la interfaz.

El resultado es una experiencia de usuario mucho más intuitiva y un juego completamente jugable de principio a fin.

Sprint 5 — Sprint Review

Este sprint se centró en la estabilidad, el pulido final y la preparación para el despliegue. Se añadió reconexión automática de jugadores, permitiendo volver a la partida si pierden conexión. El host recibe ahora un sistema de logs con eventos importantes, facilitando el control de la partida.

Se completaron todos los idiomas en todas las pantallas, se optimizó el servidor para reducir latencia y se mejoró el manejo de errores y mensajes al usuario. Finalmente, se preparó el proyecto para su despliegue en plataformas como Railway, Render o Vercel.

El resultado es una versión final estable, optimizada y lista para entrega.

2.3. Sprint Retrospectives

Sprint 1 — Sprint Retrospective

Este sprint ha sido estable y bien organizado. La planificación fue realista y permitió completar todos los objetivos previstos.

Aspectos positivos:

- *La arquitectura del backend quedó clara y modular.*
- *La comunicación en tiempo real funciona sin errores.*
- *El flujo básico del juego ya es utilizable.*

Aspectos a mejorar:

- *Realizar commits más pequeños y frecuentes.*
- *Añadir validaciones más robustas en la creación y unión de salas.*
- *Mejorar la interfaz para facilitar pruebas rápidas.*

Sprint 2 — Sprint Retrospective

La estructura del backend creada en el Sprint 1 facilitó enormemente este sprint, permitiendo implementar los roles sin complicaciones.

Aspectos positivos:

- *Flujo de roles claro y estable.*
- *Comunicación privada bien implementada.*
- *Sprint completado sin bloqueos.*

Aspectos a mejorar:

- *Añadir feedback visual más claro en la interfaz.*
- *Documentar mejor el flujo de asignación de roles.*
- *Revisar casos límite como desconexiones antes de iniciar la partida.*

Sprint 3 — Sprint Retrospective

Este ha sido el sprint más complejo hasta ahora. La lógica de roles y fases requirió mucha depuración, pero el resultado es estable y jugable.

Aspectos positivos:

- *Lógica completa del juego funcionando.*
- *Comunicación cliente-servidor fluida.*
- *Temporizadores estables y bien integrados.*

Aspectos a mejorar:

- *Añadir logs para depurar acciones de roles.*

- *Mejorar la interfaz de votación.*
- *Añadir más pruebas automáticas.*
- *Revisar interacción entre roles especiales.*

Sprint 4 — Sprint Retrospective

El rediseño de la interfaz permitió detectar pequeños errores en la lógica del juego y mejorar la cohesión entre pantallas.

Aspectos positivos:

- *Diseño coherente y fluido.*
- *Interfaz clara y temática.*
- *Se detectaron y corrigieron errores de lógica.*

Aspectos a mejorar:

- *Optimizar rendimiento en móviles.*
- *Añadir feedback visual más claro en acciones críticas.*
- *Terminar implementación de roles especiales.*
- *Revisar accesibilidad y adaptación a idiomas.*

Sprint 5 — Sprint Retrospective

Este sprint consolidó el proyecto y lo dejó en estado de producción.

Aspectos positivos:

- *Reconexión automática funcionando correctamente.*
- *Logs del host útiles y bien integrados.*
- *Interfaz pulida y coherente.*
- *Proyecto listo para despliegue.*

Aspectos a mejorar:

- *Perfeccionar la conexión automática.*
- *Añadir más pruebas unitarias.*
- *Mejorar documentación técnica del backend.*
- *Considerar añadir nuevos roles o modos de juego.*

2.4. Sprint Plannings

Sprint 1 — Base técnica del proyecto

En este primer sprint me centré en levantar toda la estructura que iba a sostener el juego. La prioridad fue conseguir que los jugadores pudieran entrar en una sala y verse entre ellos en tiempo real, porque sin esa base el resto del proyecto no tendría sentido. Implementé la creación de salas con código único y la unión mediante dicho código, asegurándome de que el servidor gestionara correctamente la persistencia de las salas y la identificación de cada jugador.

La comunicación en tiempo real mediante WebSockets fue una decisión clave: quería evitar cualquier sistema basado en peticiones periódicas porque habría generado más complejidad y peor rendimiento. Opté por WebSockets desde el principio para garantizar fluidez y escalabilidad. También preparé una interfaz mínima, muy simple, porque en esta fase lo importante era validar el flujo técnico, no la estética.

Este sprint me sirvió para definir la arquitectura del backend y comprobar que la comunicación cliente-servidor funcionaba sin problemas. Decidí mantener el código lo más modular posible para facilitar los sprints posteriores, donde sabía que la lógica iba a crecer mucho.

Sprint 2 — Roles y preparación de partida

En este sprint me enfoqué en que la partida pudiera empezar de forma ordenada. Implementé el sistema de roles automáticos, asegurándome de que cada jugador recibiera su rol de forma privada. Esta parte fue importante porque el juego depende de la información oculta, y cualquier filtración rompería la experiencia.

Decidí que el host fuera el único con permiso para iniciar la partida porque quería evitar situaciones en las que un jugador aleatorio pudiera iniciar sin que todos estuvieran listos. Esto también me permitió estructurar mejor el flujo de inicio y preparar la transición hacia las fases de noche y día.

La interfaz se actualizó para mostrar cuándo la partida estaba lista, algo que consideré necesario para que los jugadores entendieran el estado del juego sin depender de explicaciones externas. En este sprint confirmé que la base del backend del Sprint 1 había sido una buena decisión, porque permitió implementar los roles sin complicaciones.

Sprint 3 — Lógica del juego (Noche / Día)

Este sprint fue el más exigente a nivel técnico. Aquí implementé toda la lógica del juego: las acciones de los roles durante la noche, la revelación de la víctima, la votación pública durante el día y el control automático de fases con temporizadores. Me obligó a pensar en cómo sincronizar acciones privadas y públicas sin generar inconsistencias.

Decidí separar completamente la lógica de noche y día en módulos distintos porque mezclar ambas fases habría generado errores difíciles de depurar. También opté por temporizadores automáticos para que la partida avanzara sin intervención manual, buscando un ritmo de juego más dinámico y menos dependiente del host.

Durante este sprint me di cuenta de que necesitaba un sistema de logs para entender mejor qué estaba ocurriendo en cada fase, especialmente cuando varios roles actuaban a la vez. También comprobé que la interfaz necesitaba mejoras para representar las votaciones de forma más clara, algo que dejé para el sprint siguiente.

Sprint 4 — Interfaz completa

En este sprint me dediqué a transformar el juego en algo visualmente coherente y fácil de entender. Separé cada fase en pantallas independientes: lobby, roles, noche, día y resultados. Esta decisión la tomé porque quería que el jugador supiera siempre en qué parte del juego se encontraba, evitando confusiones y mejorando la experiencia general.

Añadí animaciones básicas y feedback visual para acciones como seleccionar víctima, votar o confirmar decisiones. También implementé el sistema multilingüe, que fue más complejo de lo esperado porque implicaba adaptar todos los textos y mensajes del juego. Aproveché este sprint para añadir más roles especiales y ajustar sus habilidades dentro de la interfaz.

Este rediseño me permitió detectar errores en la lógica del juego que no eran evidentes en la versión anterior. También comprobé que el rendimiento en móviles necesitaba optimización, algo que dejé para el sprint final.

Sprint 5 — Extras y pulido final

Este sprint estuvo centrado en la estabilidad y en dejar el proyecto listo para su despliegue. Implementé la reconexión automática de jugadores, una funcionalidad que consideré imprescindible para un juego online, ya que evita que un jugador pierda la partida por un fallo de conexión. También añadí un sistema de logs para el host, que permite ver eventos importantes y facilita la moderación de la partida.

Completé todos los idiomas en todas las pantallas, optimicé el servidor para reducir latencia y mejoré el manejo de errores para que los mensajes fueran más claros y útiles. El chat

integrado lo añadí como funcionalidad opcional porque mejora la experiencia, pero no es esencial para el funcionamiento del juego.

Finalmente, preparé el proyecto para su despliegue en Railway, Render o Vercel. En este punto el juego ya era estable, jugable de principio a fin y con una interfaz coherente. Este sprint me permitió cerrar el proyecto con una versión final pulida y lista para entregar.

2.5. Product Backlog

El Product Backlog de RedMoon evolucionó de forma significativa a lo largo del proyecto. Las historias de usuario se añadieron, modificaron o descartaron según las necesidades del juego y la experiencia adquirida en cada sprint.

Cambios relevantes:

- Entrada temprana de historias relacionadas con salas y WebSockets.
- Inclusión progresiva de roles y fases del juego.
- Ajustes en historias de interfaz tras detectar necesidades de claridad visual.
- Añadido de reconexión y logs en sprints finales por estabilidad.
- Descarte de funcionalidades que no aportaban al MVP (por ejemplo, modos adicionales).

El backlog se mantuvo vivo y adaptado a la realidad del desarrollo, reflejando la naturaleza iterativa de Scrum.

2.6. Sprint Backlog

Cada sprint contó con su Sprint Backlog correspondiente, donde se desglosaron las historias seleccionadas en tareas concretas. El equipo utilizó GitHub Projects para gestionar el estado de cada tarea (Todo, In Progress, To Test, Done), lo que permitió visualizar el progreso y evitar solapamientos.

2.7. Descripción del contenido del repositorio en GitHub

El repositorio de RedMoon utilizó las siguientes funcionalidades de GitHub:

- **Issues** para registrar historias de usuario y tareas.
- **Projects (Kanban)** para organizar el trabajo por sprints.
- **Wiki** para documentar sprints, decisiones y arquitectura.
- **Branches** para desarrollar funcionalidades sin afectar la rama principal.
- **Releases** para entregar cada sprint y la release final.

3. Otras tareas de gestión del proyecto

3.1. Gestión de riesgos

El equipo elaboró un plan de riesgos que incluía:

- Riesgos técnicos (fallos de sincronización, errores de roles).
- Riesgos organizativos (falta de comunicación, mala estimación).
- Riesgos académicos (exámenes, carga de otras asignaturas).

Durante el proyecto se revisaron los riesgos y se ajustaron según la situación real. Por ejemplo, se detectó que los exámenes afectaban más de lo previsto y se adaptaron las reuniones y la planificación.

.

3.3. Revisión de la gestión de riesgos

Gestión de Riesgos del Proyecto RedMoon

La gestión de riesgos en RedMoon ha sido un proceso continuo y especialmente relevante debido a que el proyecto ha sido desarrollado íntegramente por una sola persona. Esto implica que cualquier error, retraso o mala decisión técnica tiene un impacto directo y completo sobre el avance del proyecto. Por ello, aunque no se utilizó una metodología formal desde el primer día, la identificación y mitigación de riesgos se convirtió en una parte esencial del desarrollo.

A continuación se presenta la gestión de riesgos realizada durante el proyecto, adaptada a la realidad de un desarrollo individual.

1. Identificación inicial de riesgos

Al comenzar el proyecto, se detectaron varios riesgos que podían comprometer el avance del desarrollo. Estos riesgos se clasificaron en tres categorías principales:

Riesgos técnicos

- Fallos en la comunicación en tiempo real mediante WebSockets.
- Errores en la sincronización de roles y fases del juego.
- Complejidad elevada de la lógica de noche/día.
- Posible pérdida de coherencia entre cliente y servidor.
- Sobrecarga del backend al gestionar múltiples jugadores.
- Dificultad para depurar acciones privadas de roles.

Riesgos organizativos (derivados de trabajar solo)

- Falta de revisión externa del código.
- Acumulación de tareas sin dividir correctamente.
- Riesgo de estancarse en un problema técnico sin apoyo.
- Sobrecarga mental al tener que abarcar backend, frontend, diseño y pruebas.
- Dificultad para mantener una planificación realista sin feedback externo.

Riesgos académicos

- Coincidencia con exámenes y entregas de otras asignaturas.
- Reducción de tiempo disponible en semanas críticas.
- Fatiga acumulada por compaginar múltiples proyectos.

2. Evolución de los riesgos durante el proyecto

A medida que avanzaron los sprints, varios riesgos se hicieron evidentes y obligaron a ajustar la estrategia de desarrollo.

2.1. Riesgos técnicos

WebSockets y sincronización: En el Sprint 1 se comprobó que la comunicación en tiempo real era más delicada de lo previsto. Para evitar errores futuros, se decidió separar la lógica de salas, jugadores y mensajes en módulos independientes.

Roles y fases: En el Sprint 3 la lógica de noche/día mostró ser el punto más complejo del proyecto. La falta de revisión externa aumentó el riesgo de errores silenciosos, por lo que se añadieron logs internos y pruebas manuales exhaustivas.

Interfaz por fases: En el Sprint 4 se detectó que la interfaz podía ocultar errores de lógica si no se probaba cada transición cuidadosamente. Se decidió revisar cada pantalla de forma aislada antes de integrarla en el flujo completo.

2.2. Riesgos organizativos

Acumulación de tareas: En los primeros sprints las tareas eran demasiado grandes. Al trabajar solo, esto generaba bloqueos prolongados. A partir del Sprint 2 se empezó a dividir cada historia en subtareas pequeñas y manejables.

Estancamiento técnico: En el Sprint 3 surgieron problemas complejos en la sincronización de roles. Sin compañeros para consultar, se adoptó la estrategia de documentar cada módulo y aislar errores mediante logs detallados.

Sobrecarga mental: En los sprints finales, la combinación de backend, frontend, diseño y pruebas generó fatiga. Para mitigarlo, se estableció una rutina de trabajo más estructurada y se priorizaron tareas críticas.

2.3. Riesgos académicos

Exámenes: En los Sprints 4 y 5 los exámenes afectaron al ritmo de trabajo. Se ajustó la planificación para evitar tareas grandes en esas semanas y se priorizó la estabilidad del proyecto.

3. Medidas de mitigación adoptadas

A lo largo del proyecto se implementaron diversas acciones para reducir el impacto de los riesgos:

- **Modularización del backend** para evitar errores en la lógica de roles y fases.
- **División de tareas grandes en subtareas pequeñas** para evitar bloqueos.
- **Uso intensivo de logs internos** para depurar acciones privadas de roles.
- **Pruebas manuales exhaustivas** en cada fase del juego.
- **Documentación incremental** para no perder el contexto de decisiones técnicas.
- **Planificación adaptada a semanas de exámenes.**
- **Revisión manual de merges** para evitar conflictos automáticos.
- **Separación estricta entre frontend y backend** para facilitar la depuración.
- **Uso de prototipos rápidos** para validar ideas antes de implementarlas completamente.

Estas medidas permitieron mantener el proyecto estable incluso sin apoyo externo.

4. Revisión de riesgos durante los sprints

Cada sprint incluyó una revisión informal de los riesgos, centrada en:

- Qué problemas habían surgido.
- Qué decisiones técnicas habían funcionado.
- Qué tareas habían sido demasiado grandes.
- Qué partes del código necesitaban refactorización.
- Qué riesgos aumentaban y cuáles disminuían.

Ejemplos:

- El riesgo de “tareas demasiado grandes” disminuyó notablemente a partir del Sprint 3.
- El riesgo de “errores en roles especiales” aumentó en el Sprint 4, lo que llevó a revisar la lógica de interacción entre roles.
- El riesgo académico de “exámenes” se mantuvo constante y se gestionó mediante planificación adaptada.

5. Evaluación final de la gestión de riesgos

La gestión de riesgos evolucionó desde un enfoque inicial muy básico hacia un sistema más consciente y adaptado a la realidad de un desarrollo individual. Aunque no se utilizó una herramienta formal de análisis cuantitativo, el seguimiento continuo permitió:

- Reducir errores técnicos graves.
- Evitar bloqueos prolongados en tareas complejas.
- Mantener la estabilidad del proyecto sin apoyo externo.
- Aumentar la productividad en sprints finales.
- Completar el proyecto dentro de los plazos previstos.

En conclusión, la gestión de riesgos fue una parte esencial del éxito de RedMoon. La naturaleza individual del proyecto hizo que cada riesgo tuviera un impacto mayor, pero también permitió una toma de decisiones rápida y coherente. El resultado final demuestra que la mitigación aplicada fue efectiva y que el proyecto se desarrolló de forma estable y controlada.